

Deep Learning Optimized on Jean Zay

Dataset optimization

Storage spaces and data format

cnrs IDRIS

CC BY NC SA

Dataset optimization

Main bottlenecks ◂

Data storage ◂

Data format ◂

Dataset format ◂

Focus on I/O requests and data decoding

jpg

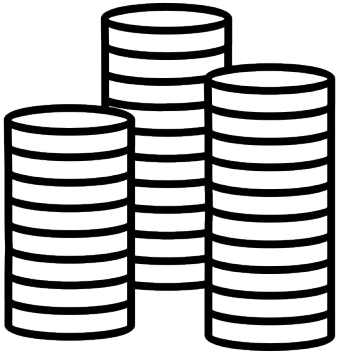


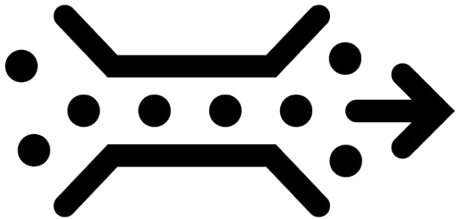
I/O request + decoding

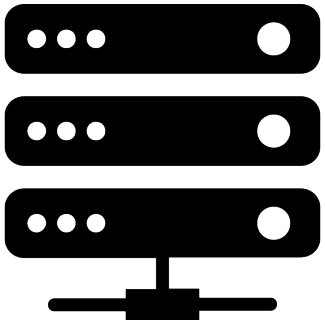
Torch tensor

```
tensor([[[144, 145, 145,  ...
         [144, 144, 145,  ...
         [144, 144, 145,  ...
         ...,
         [ 18,  39,  55,  ...
         [  6,  35,  54,  ...
         [  0,  14,  42,  ...
```

CPU
RAM







Storage disks

Interconnection network
OmniPath/InfiniBand

CPU workers

Focus on I/O requests and data decoding

jpg

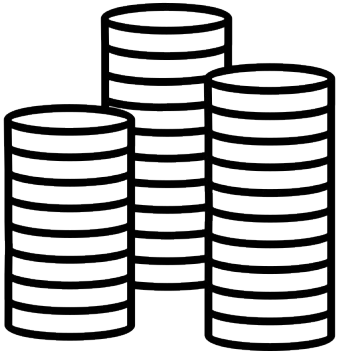


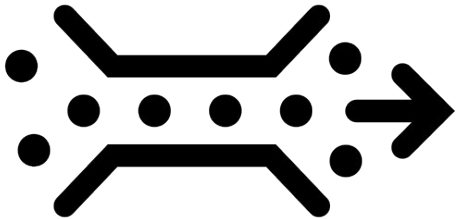
I/O request + decoding

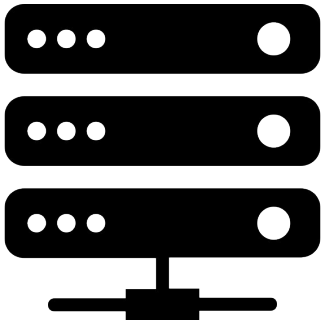
Torch tensor

```
tensor([[[144, 145, 145,  ...
         [144, 144, 145,  ...
         [144, 144, 145,  ...
         ...,
         [ 18,  39,  55,  ...
         [  6,  35,  54,  ...
         [  0,  14,  42,  ...
```

CPU
RAM







Storage disks

Interconnection network
OmniPath/InfiniBand

CPU workers



Read/write performance

Shared bandwidth

Decoding performance

Focus on I/O requests and data decoding

jpg



I/O request + decoding

Torch tensor

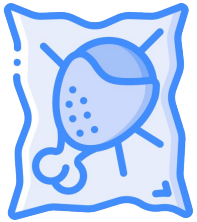
```
tensor([[[144, 145, 145,  ...
         [144, 144, 145,  ...
         [144, 144, 145,  ...
         ...,
         [ 18,  39,  55,  ...
         [  6,  35,  54,  ...
         [  0,  14,  42,  ...
```

CPU
RAM











Read/write performance

Shared bandwidth

Decoding performance

Focus on I/O requests and data decoding

The efficiency of I/O operations and data decoding depends on both the **data format** and the **storage strategy**.

- **Where should I store my dataset?**
- **Which format for my data?**
- **Which format for my dataset?**

Dataset optimization

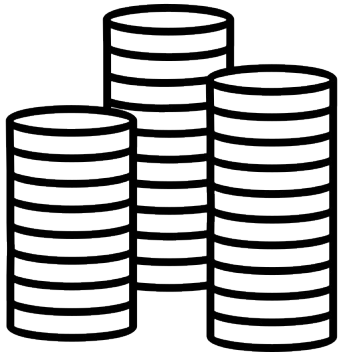
Main bottlenecks ◂

Data storage ◂

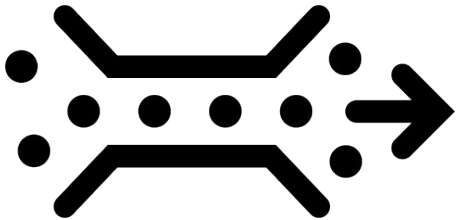
Data format ◂

Dataset format ◂

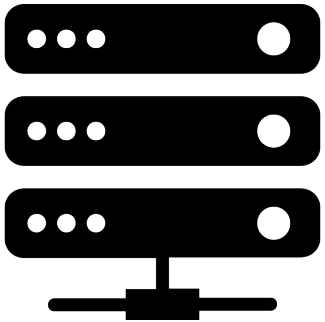
Focus on I/O requests and data decoding



Storage disks



Interconnection network
OmniPath/InfiniBand



CPU workers



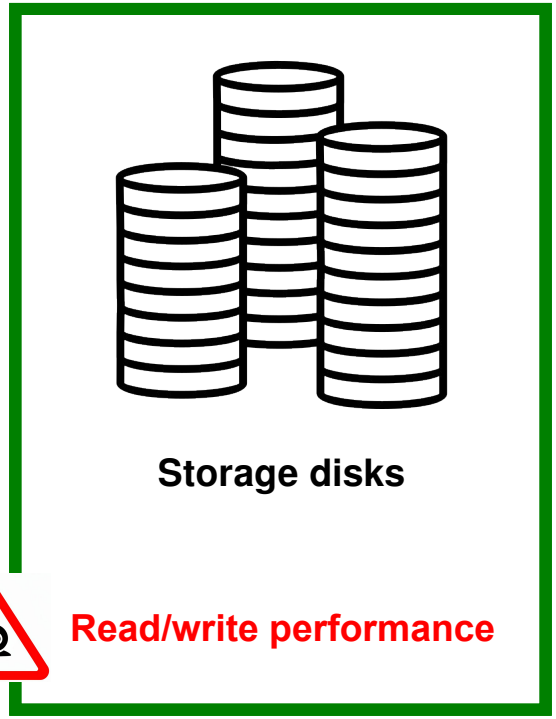
Read/write performance

Shared bandwidth

Decoding performance

Where should I store my dataset?

Various storage disk spaces

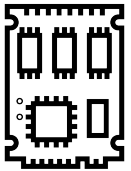


WORK
Rotating disks



300-350 GB/s

SCRATCH
Full Flash SSD



1.1-1.5 TB/s

NVMe
Local disk
(test configuration)



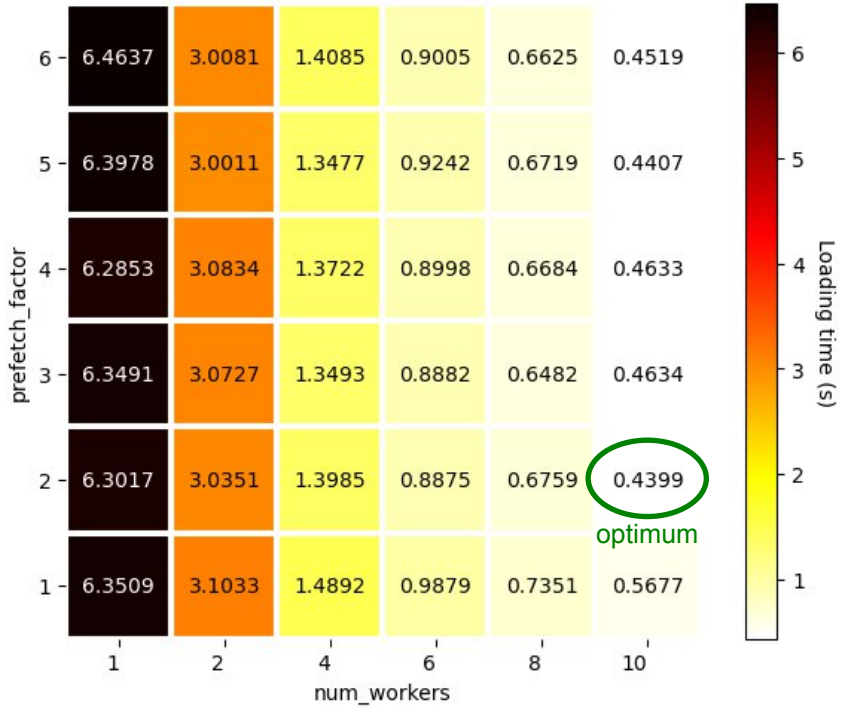
~SSD

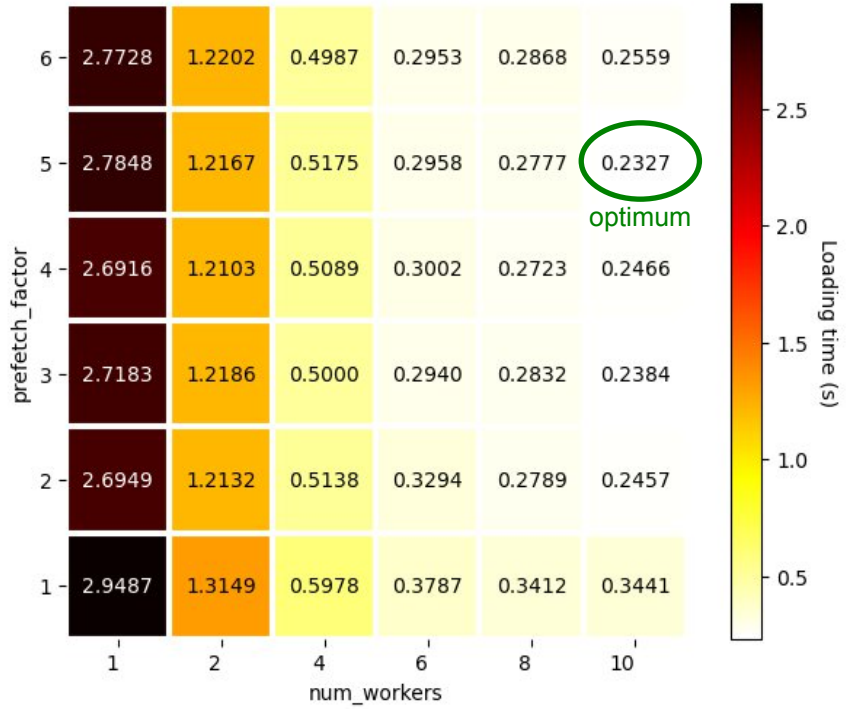
Where should I store my dataset?

Various storage disk spaces

dlojz.py - 50 iterations - test partition (A100 PCIe)

Historical benchmark - 2023 infrastructure. These absolute results should not be interpreted as current performance.





WORK

300-350 GB/s

÷ 2

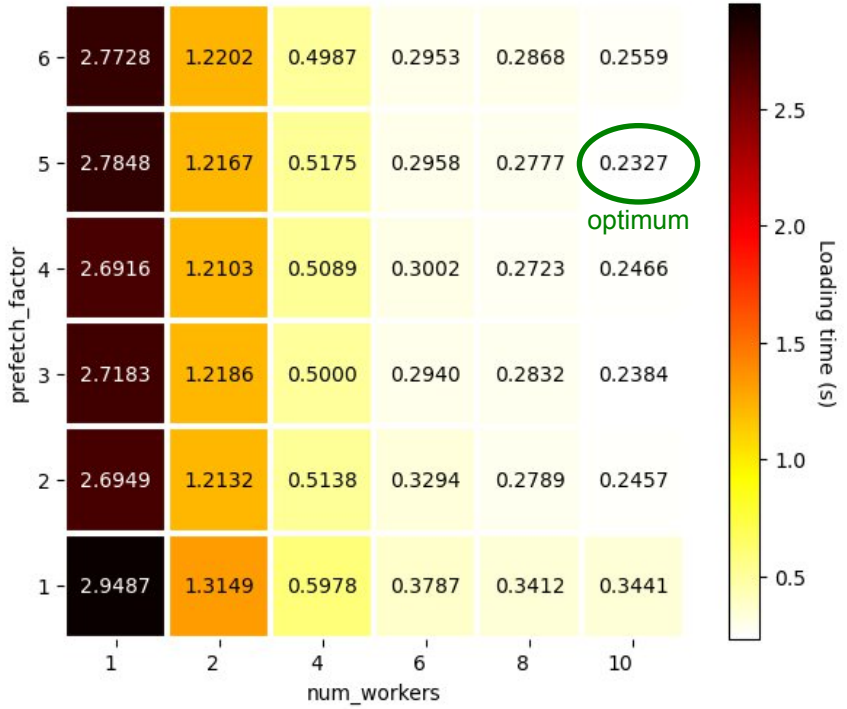
SCRATCH

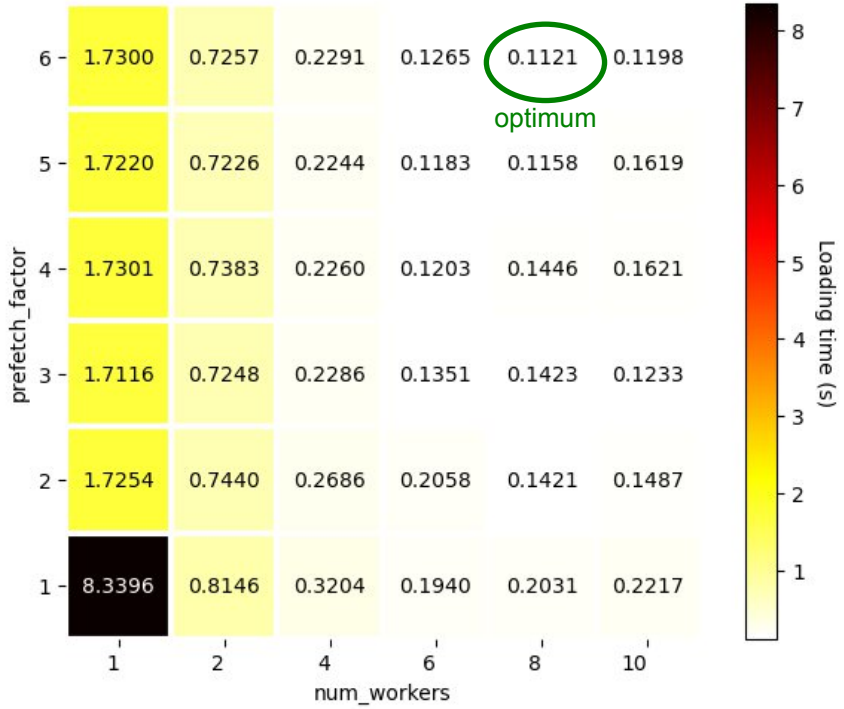
1.1-1.5 TB/s

Various storage disk spaces

dlojz.py - 50 iterations - test partition (A100 PCIe)

Historical benchmark - 2023 infrastructure. These absolute results must not be interpreted as current performance.





SCRATCH

1.1-1.5 TB/s

÷ 2

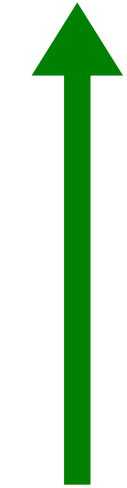
NVMe

~SSD

Various storage disk spaces

I/O speed



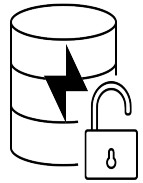




- **NVMe**
  - ✓ Best I/O performance
  - – You need to copy your dataset to the local disk first, which can take a very long time
  - – This solution is not suitable for supercomputer scale, so it is not available to users
- **SCRATCH**
  - ✓ Best possible I/O performance on Jean Zay
  - ✓ Very large quota (safety quota: ~500 TB / 150M inodes per project)
  - – 30 days file lifespan
- **WORK**
  - – Worst performance (but not bad!)
  - – Only 5 TB and 500k inodes per project
  - ✓ No automatic deletion procedure

Various storage disk spaces

- **Don’t forget the DSDIR for public datasets or models**!
  - SSD storage technology (similar to the SCRATCH but older generation)
  - Storage space accessible to all users
  - Downloads are handled by user support upon request



- **What about the STORE?**
  - Magnetic tape storage (with a cache on rotating disk)
  - 50 TB and 100k inodes per project
  - Intended to receive a small number of very large files (up to 10 TiB per file) → **archives**
  - Not accessible from the compute nodes (very high read/write latency)



Good practices

- Pool resources in the **DSDIR** if possible
- If you don’t need I/O performance (when building your workflow, debugging, or simply if I/O is not a bottleneck), store your dataset in the **WORK** if it is not too large
- If you need I/O performance or if the dataset is too large, store your dataset in the **SCRATCH**
- Keep a duplicate of your dataset as an archive in the **STORE**

Dataset optimization

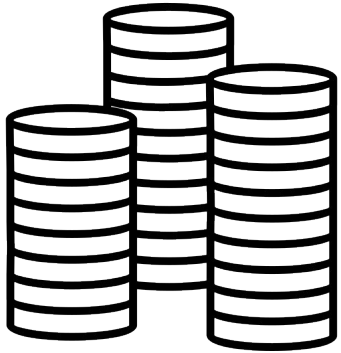
Main bottlenecks ◂

Data storage ◂

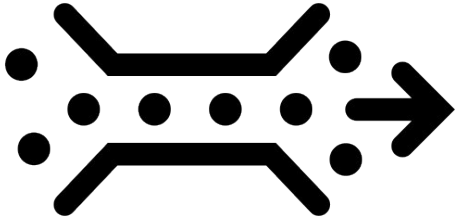
Data format ◂

Dataset format ◂

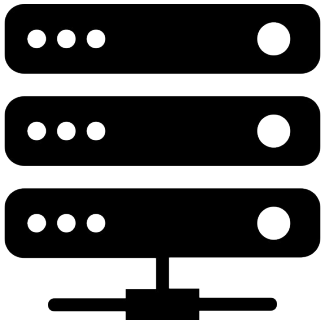
Focus on I/O requests and data decoding



Storage disks



Interconnection network
OmniPath/InfiniBand



CPU workers



Read/write performance

Shared bandwidth

Decoding performance

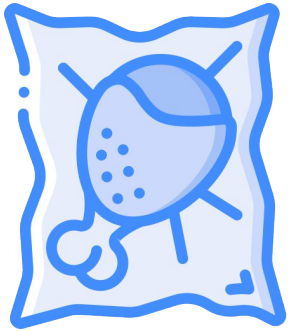
Which format for my data?

Data format



Binary format: Pickle, HDF5,...
Decoded more quickly, takes more space

- ✓ Decoder performance
- – Storage volume



Compressed format: jpeg, png,...
Decoded more slowly, takes less space

- – Decoder performance
- ✓ Storage volume

Dataset optimization

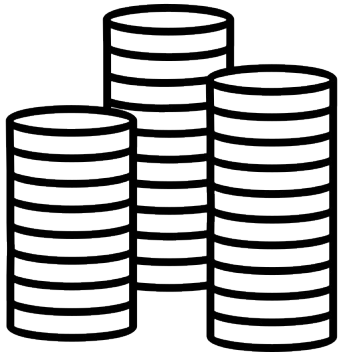
Main bottlenecks ◂

Data storage ◂

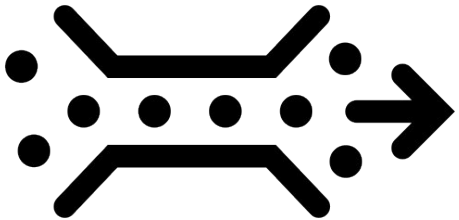
Data format ◂

Dataset format ◂

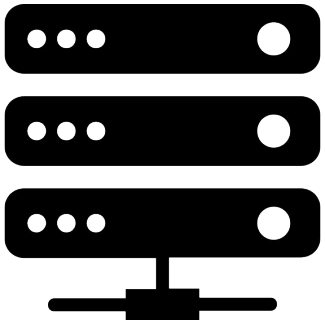
Focus on I/O requests and data decoding



Storage disks



Interconnection network
OmniPath/InfiniBand



CPU workers



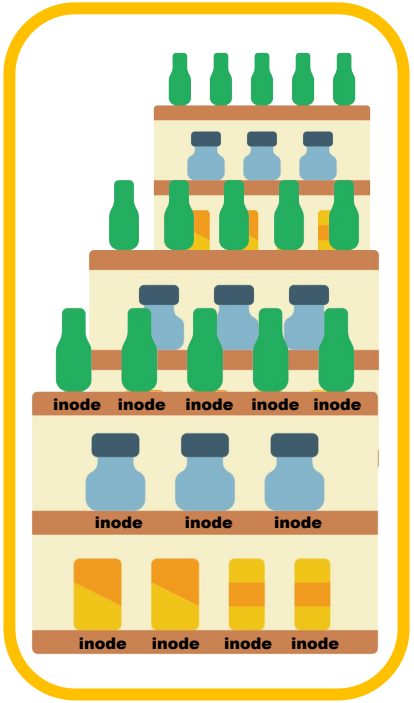
Read/write performance

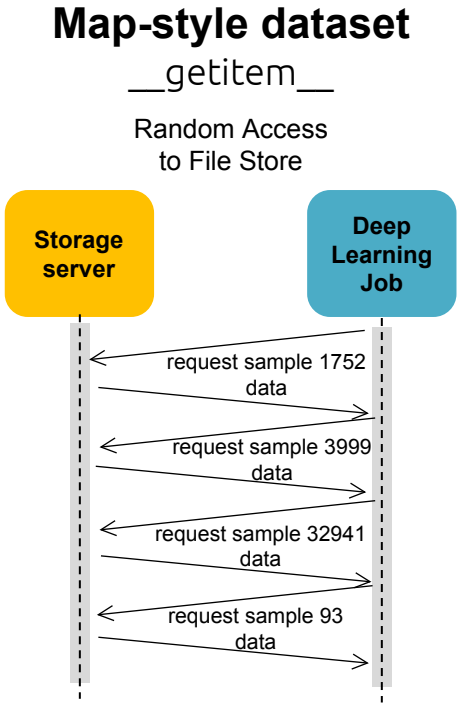
Shared bandwidth

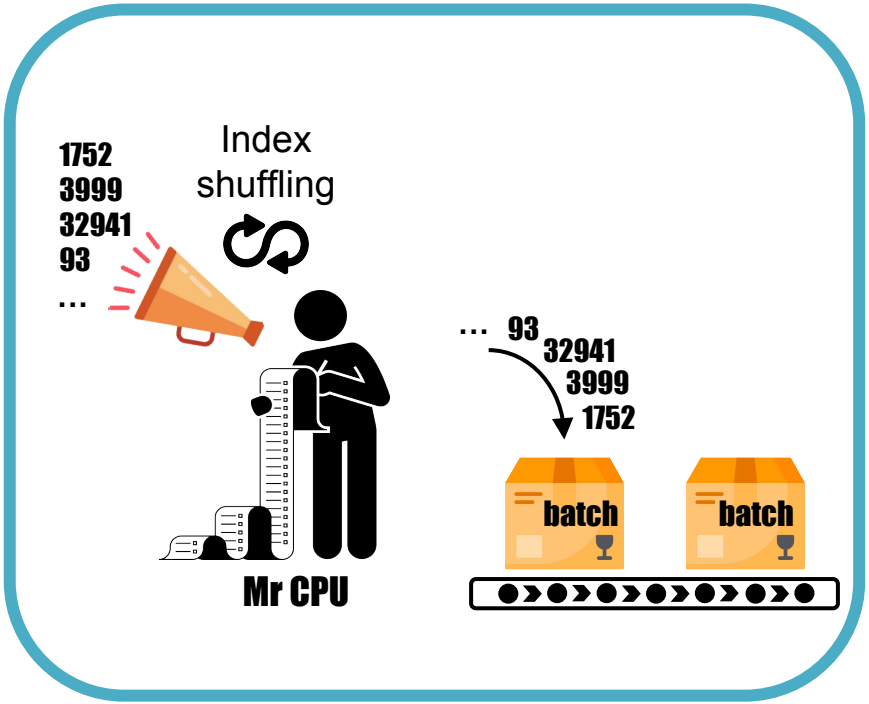
Decoding performance

Which format for my dataset?

Intuitive way: 1 sample = 1 file







Pros: Easy to handle, random access possible

Cons: Lots of inodes, lots of I/Os

Too many inodes are an issue

Error: Disk quota exceeded

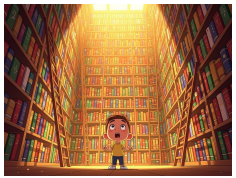


- The WORK quota per project is 5 TB / **500k inodes**
- The SCRATCH safety quota per project is ~500 TB / **150M inodes**

+ Parallel file system does not handle well:

- Intensive I/O workloads involving many small files
- A large number of files in a single directory





Too many inodes are an issue

- IDRIS recommendation on Jean Zay: **less than 100k inodes per directory**

- Repository limitations and recommendations on HuggingFace Hub 

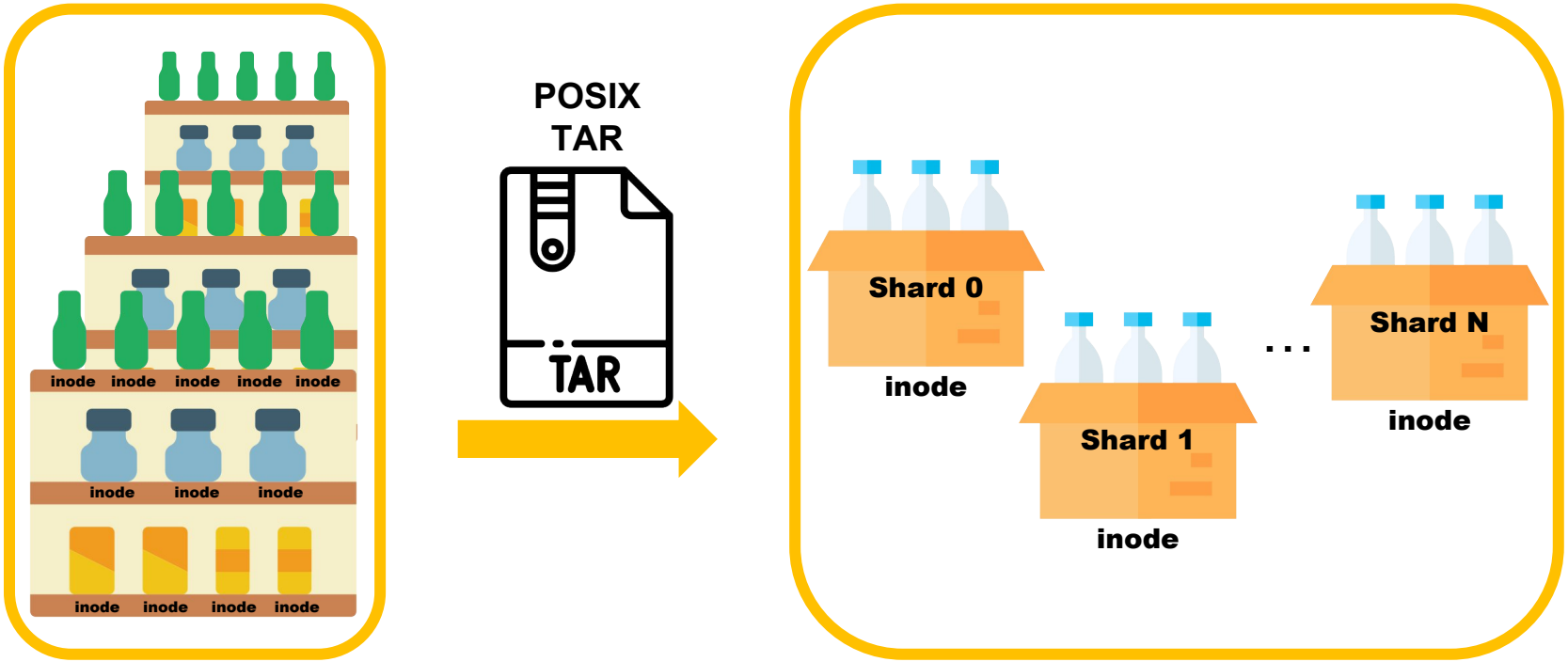
Characteristics	Recommended	Tips
Files per repo	<100k	merge data into fewer files
Entries per folder	<10k	use subdirectories in repo

https://huggingface.co/docs/hub/storage-limits#repository-limitations-and-recommendations

+ for large datasets, **WebDataset**-like format is recommended

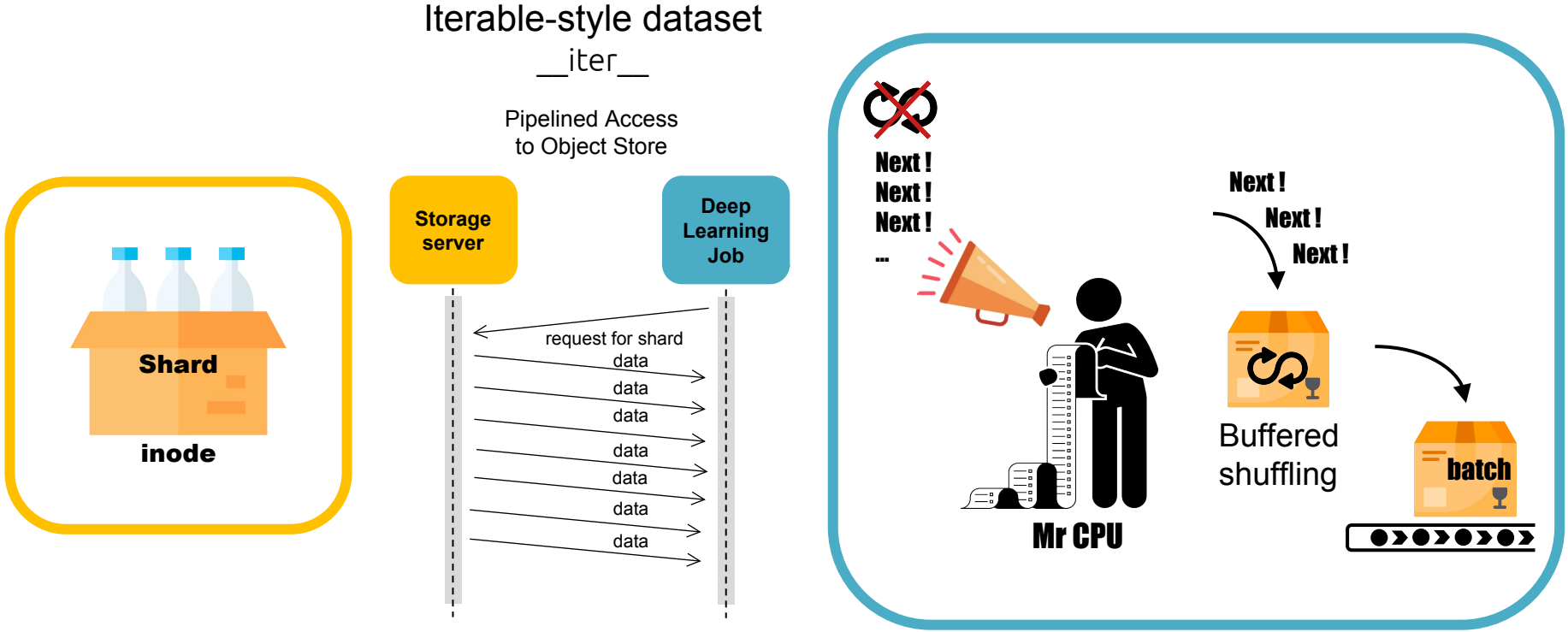
(TFRecord, Apache Parquet, HDF5, Zarr,...)

WebDataset format – Gathering data



Icon by Magnific

WebDataset format – Iterable dataset

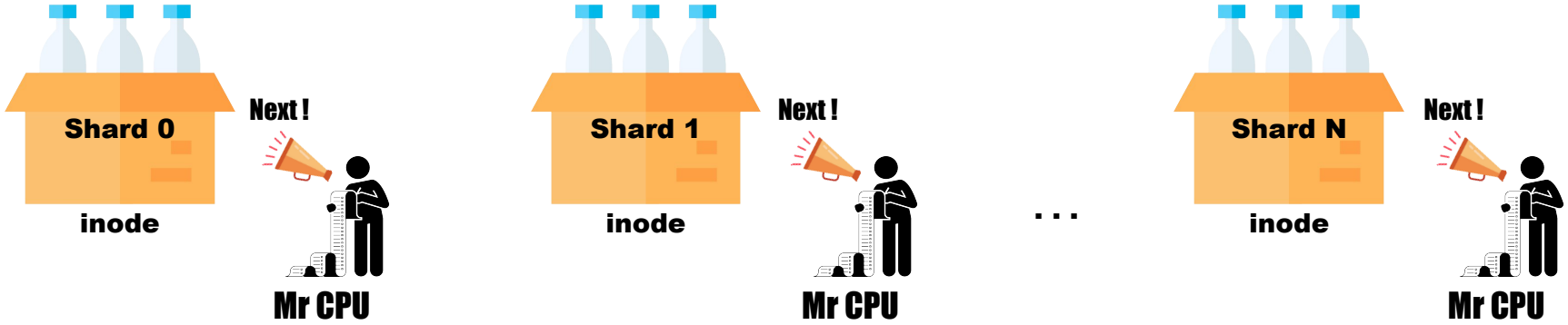


Pros: Fewer I/Os, fewer inodes

Cons: Preprocessing, difficult to shuffle or distribute, unknown dataset length

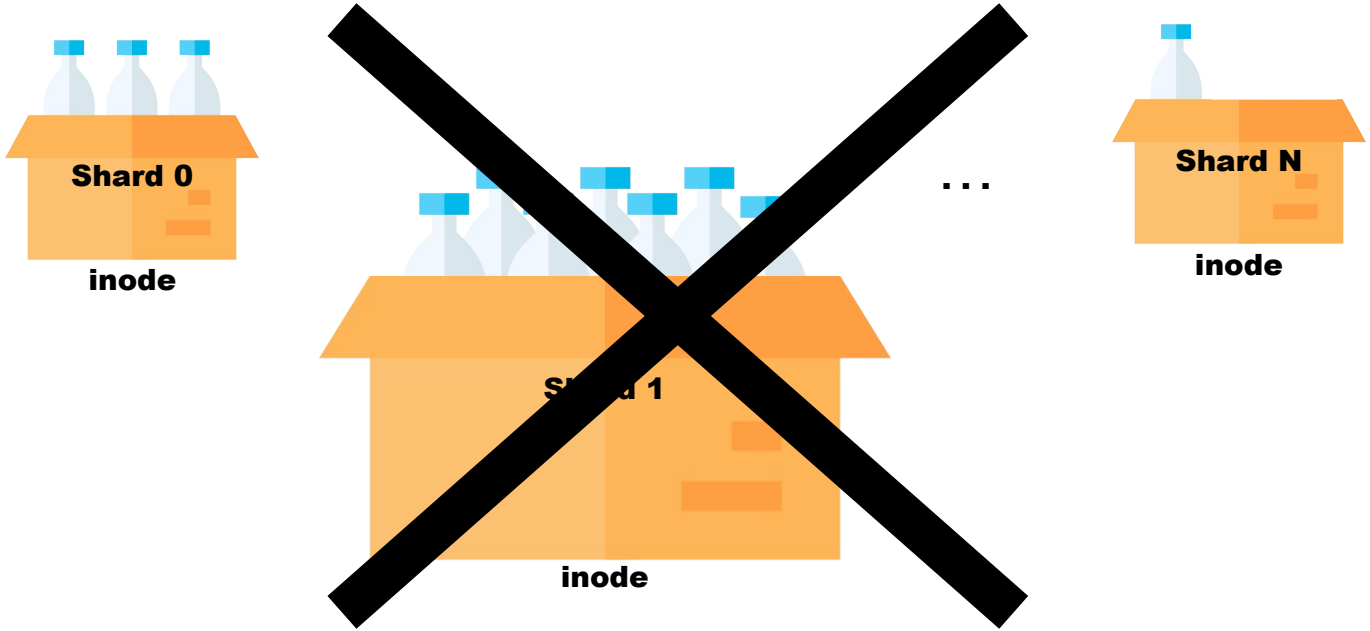
WebDataset format - Sharding

Sharding is necessary to benefit from parallel implementation
(DataLoader multi-processing and Distributed Data Parallelism).



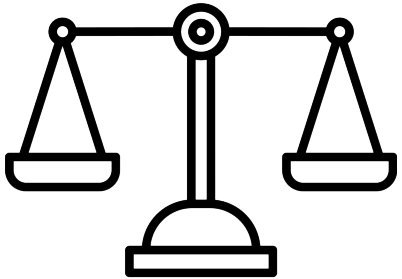
The number of shards should be a multiple of the number of tasks/GPUs.

WebDataset format - Sharding

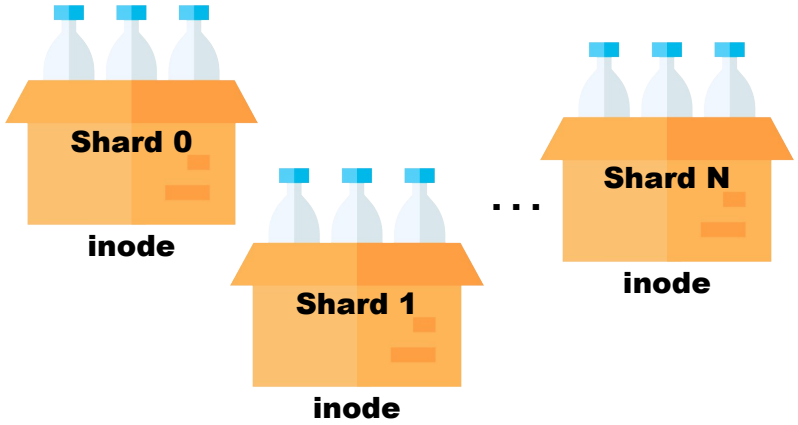


Samples must be evenly distributed among the shards to balance the workload between processes.

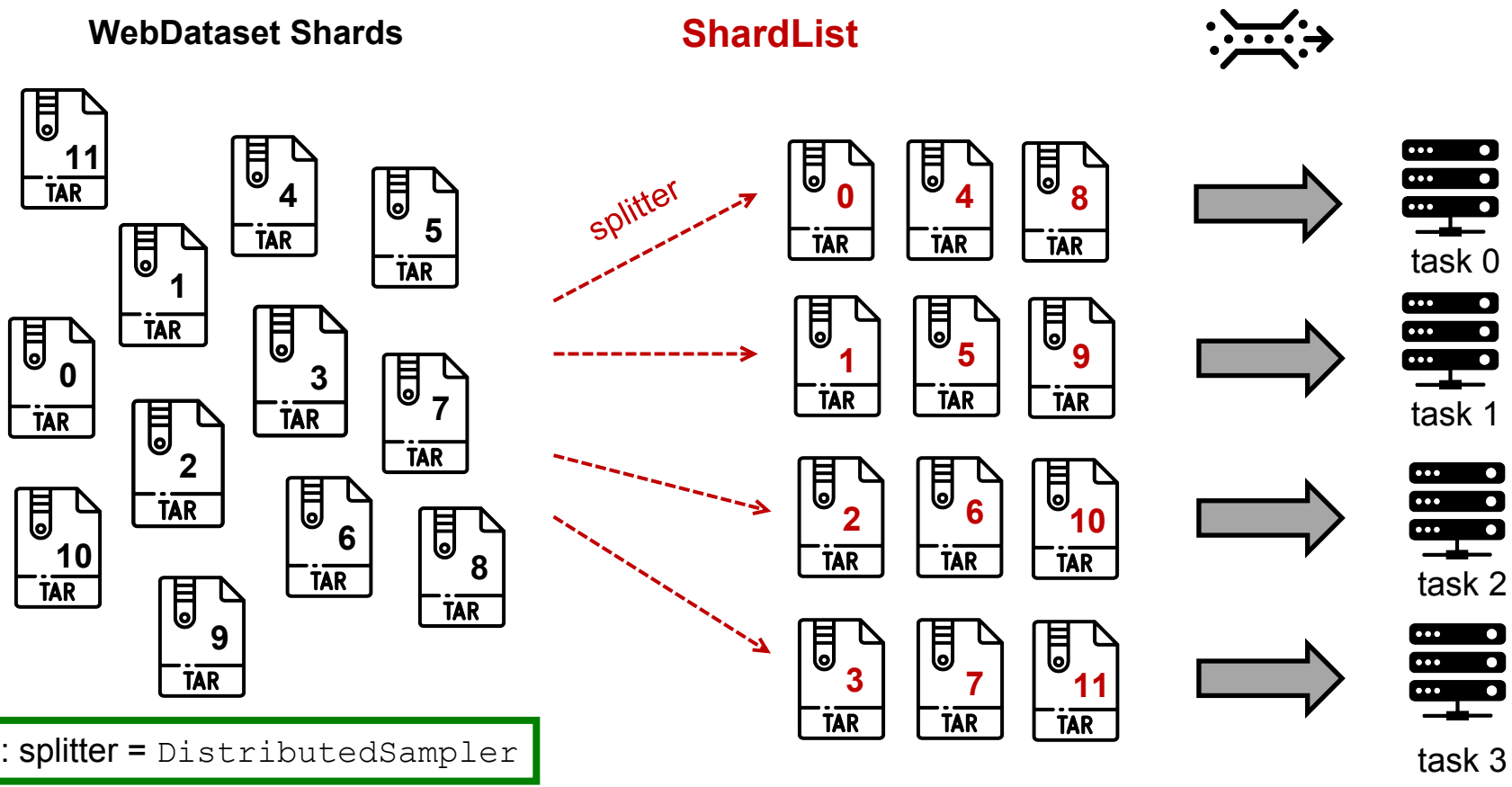
WebDataset format - More or fewer shards?



	More shards	Fewer shards
Large-scale distribution	👍	👎
CPU RAM usage	👍	👎
Inode quota	👎	👍
Number of I/O requests	👎	👍

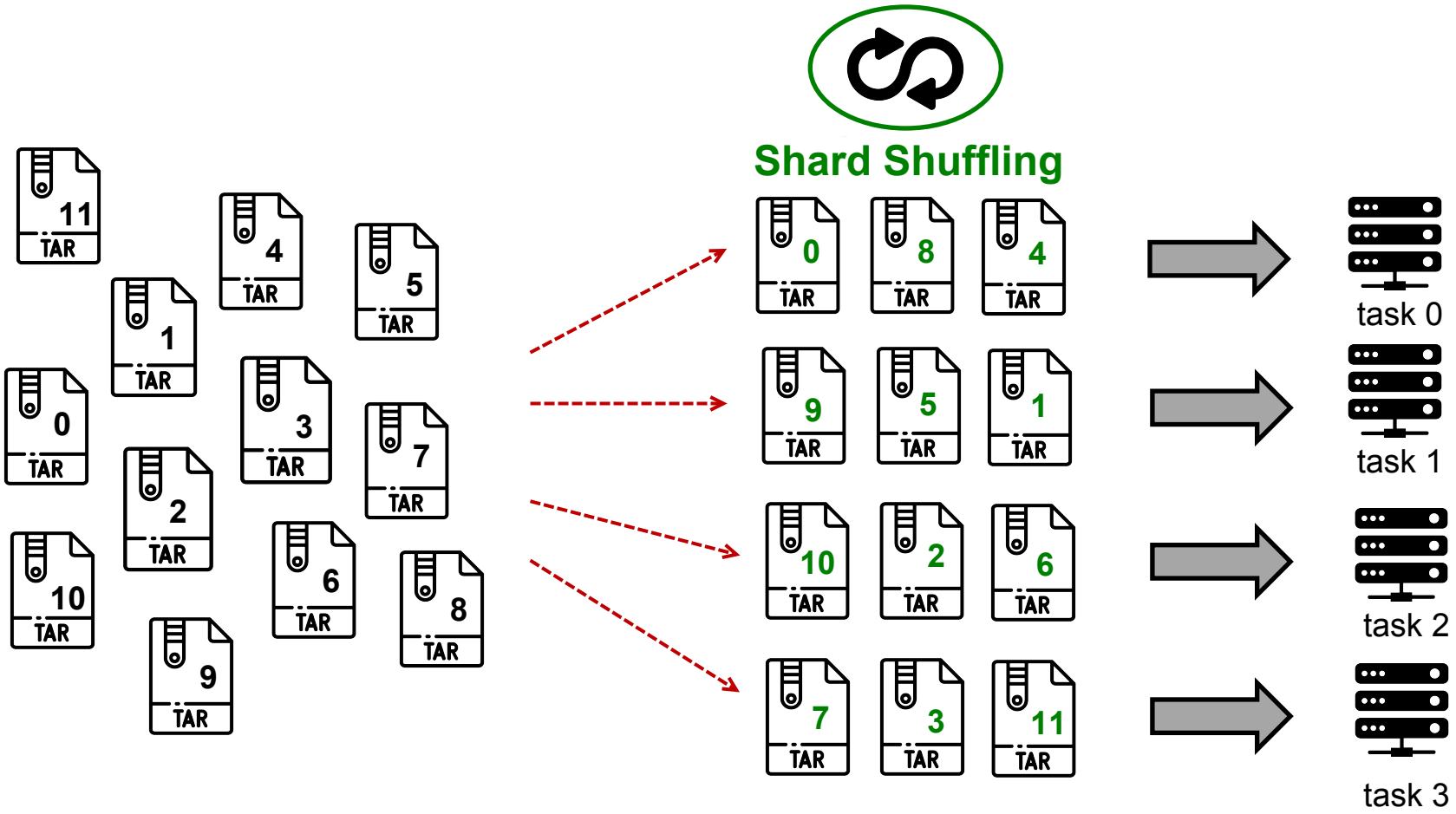


WebDataset – Multiworker sharding

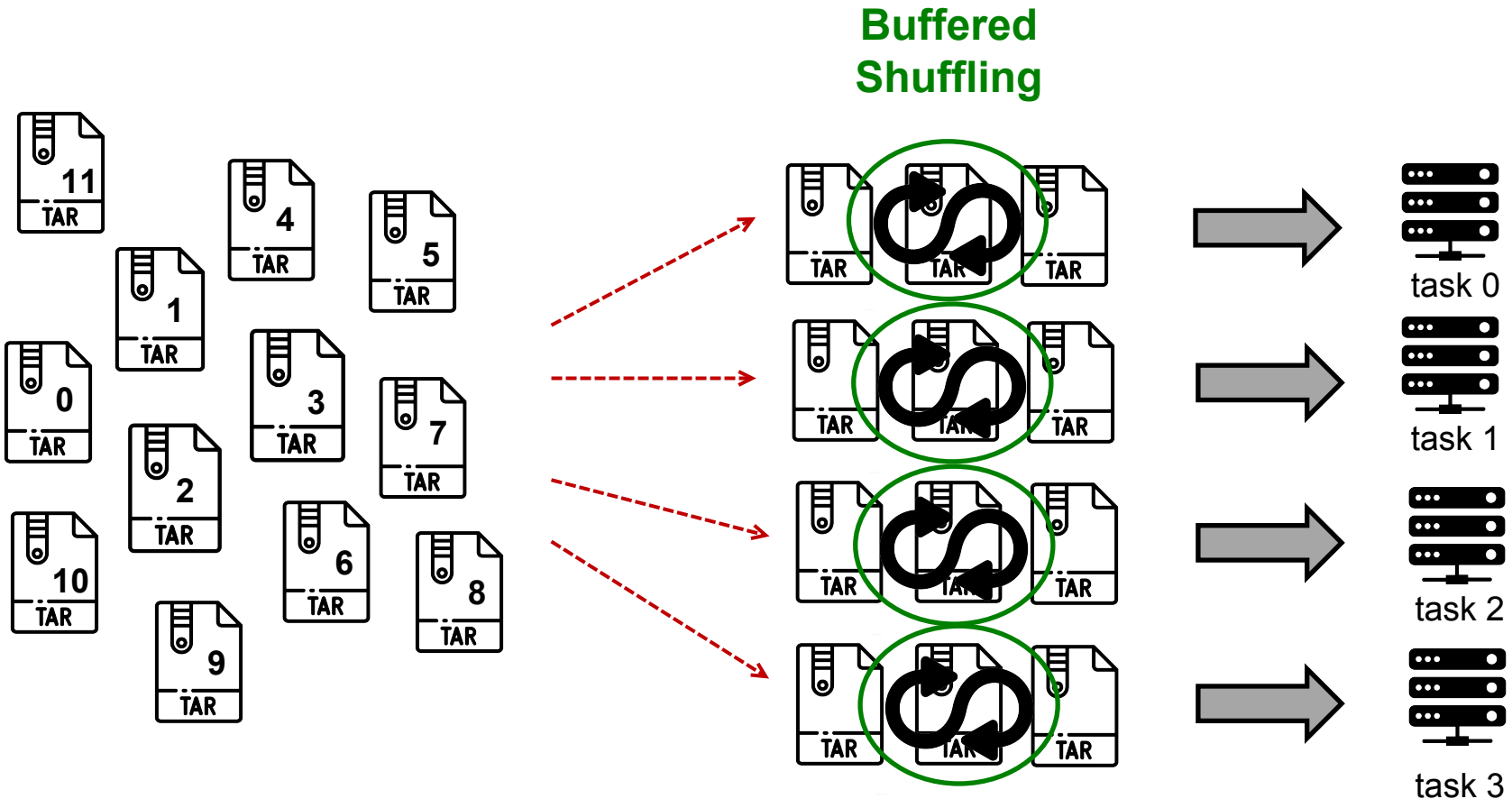


NB: splitter = `DistributedSampler`

WebDataset - Shuffling

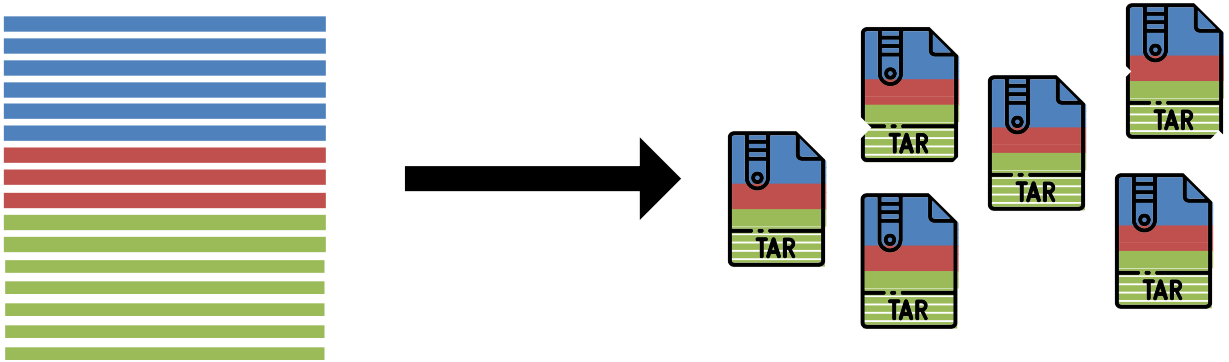


WebDataset - Shuffling



WebDataset - Preprocessing

- Distribute the samples as **evenly** as possible among the shards.
- Choose the number of shards **based on the number of GPUs and DataLoader workers** you will use.
- Distribute the samples so that each shard contains a **representative part of the dataset.**



+ Converting data before creating the archives to improve decoding performance?





WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

distribute shards among processes
(~DistributedSampler)

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



shuffling shards indexes per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

shuffling samples per process

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

} description of shard content

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

} transforming and batching

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

define len(train_dataset)

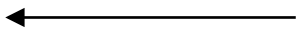
WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



batching handled by WebDataset class

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

usual DataLoader args

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```

drop_last equivalent

WebDataset - Implementation

```
import webdataset as wds

def my_splitter(paths):
    paths = list(paths)
    return paths[idr_torch.rank::idr_torch.world_size]

paths = os.environ['DSDIR']+'/imagenet/webdataset/imagenet_train-{000000..000127}.tar'
train_dataset_len = 1281167
train_dataset = wds.WebDataset(paths, nodesplitter=my_splitter, shardshuffle=True)\
                                .shuffle(1000)\
                                .decode("pil")\
                                .to_tuple('jpg', 'cls')\
                                .map_tuple(transform, lambda x: x)\
                                .batched(mini_batch_size)\
                                .with_length(train_dataset_len)

nbatches = train_dataset_len // global_batch_size
train_loader = wds.WebLoader(train_dataset, batch_size=None,\
                             num_workers=num_workers,\
                             persistent_workers=persistent_workers,\
                             pin_memory=pin_memory,\
                             prefetch_factor=prefetch_factor\
                             ).slice(nbatches)
train_loader.length = nbatches
```



define len(train_loader)

WebDataset - Performance test

I/O loop test: timing I/O requests and decoding operations

```
for i, (images,labels) in enumerate(loader):
    assert images.size()[0] == batch_size
```

Test performed on 1 V100 GPU (32GB)

- Loop over 10 iterations
- Calculation-free iterations & Minimum data transformation





Mean iteration time

WebDataset - Performance test

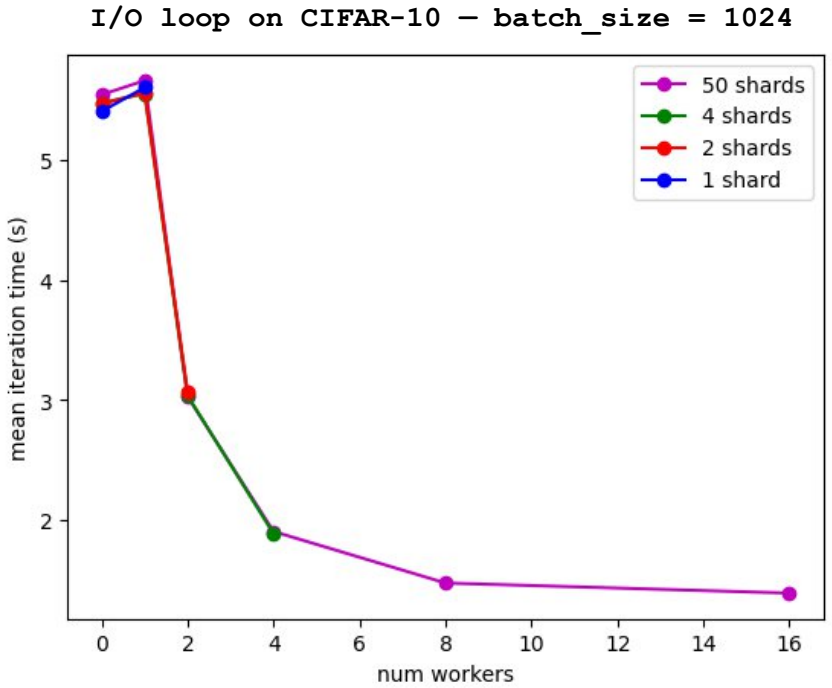
🤔 Can I work with only 1 shard?

Benchmark 1

Dataset: CIFAR-10 (50k images, no transformation)

Format: WebDataset, from 1 to 50 shards

Run: I/O loop while increasing the number of workers



- Sharding is necessary to benefit from parallel implementation (DataLoader multi-processing).

WebDataset - Performance test

What do I get out of using WebDataset?

Benchmark 2

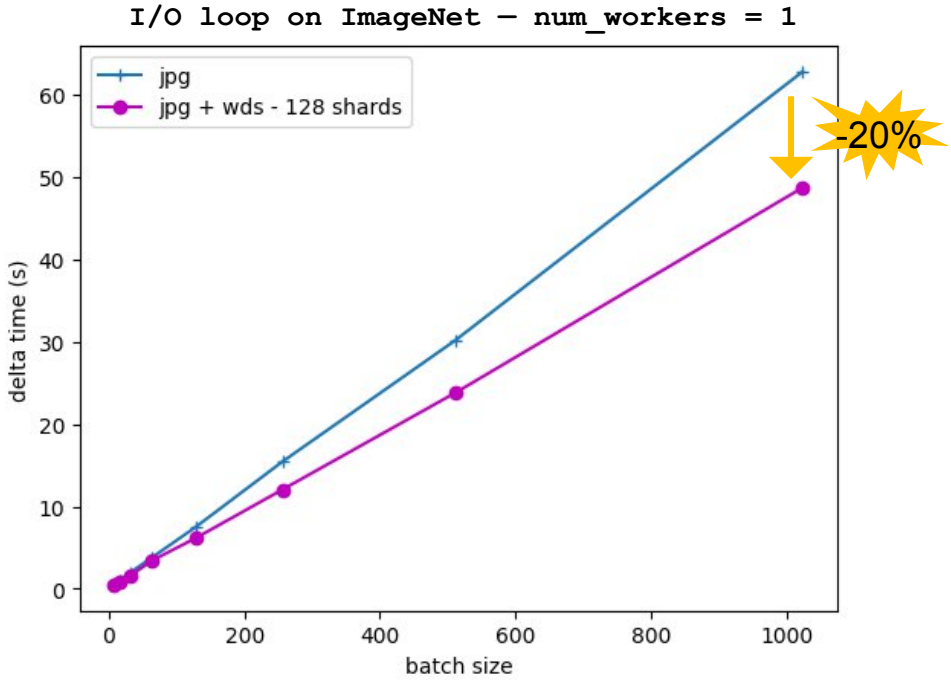
Dataset: ImageNet

- ~1.3M images
- *CenterCrop* is needed since images have variable sizes

Format:

- Original format: 1.3M JPG files (~140 GB)
- WebDataset with 128 shards (~213 GB)

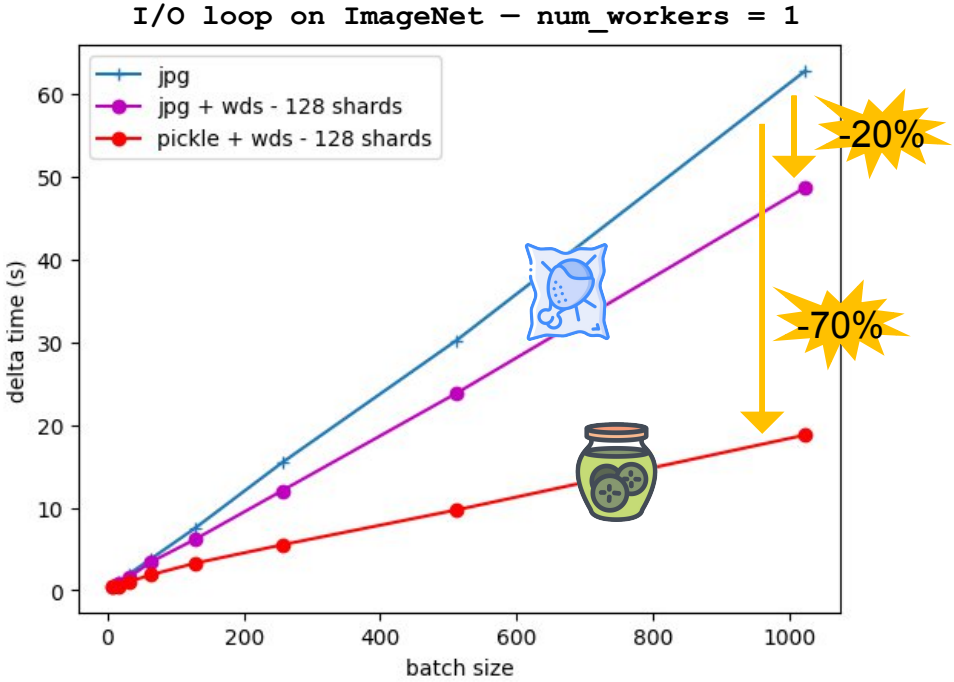
Run: I/O loop while increasing the batch size



- 20% I/O speedup (with 1 worker)
- From 1.3M to 128 inodes for a similar storage volumetry

WebDataset - Performance test

🤔 Creating a WebDataset is time-consuming, why not store the data in binary format as well?



Benchmark 2 (extended)

Format:

- Original format: 1.3M JPG files (~140 GB)
- WebDataset with 128 shards (~213 GB)
- The same but with decoded samples (~826 GB)

- 70% I/O speedup (with 1 worker)
- From 1.3M to 128 inodes, but from 140 to 826 GB!

WebDataset - Performance test

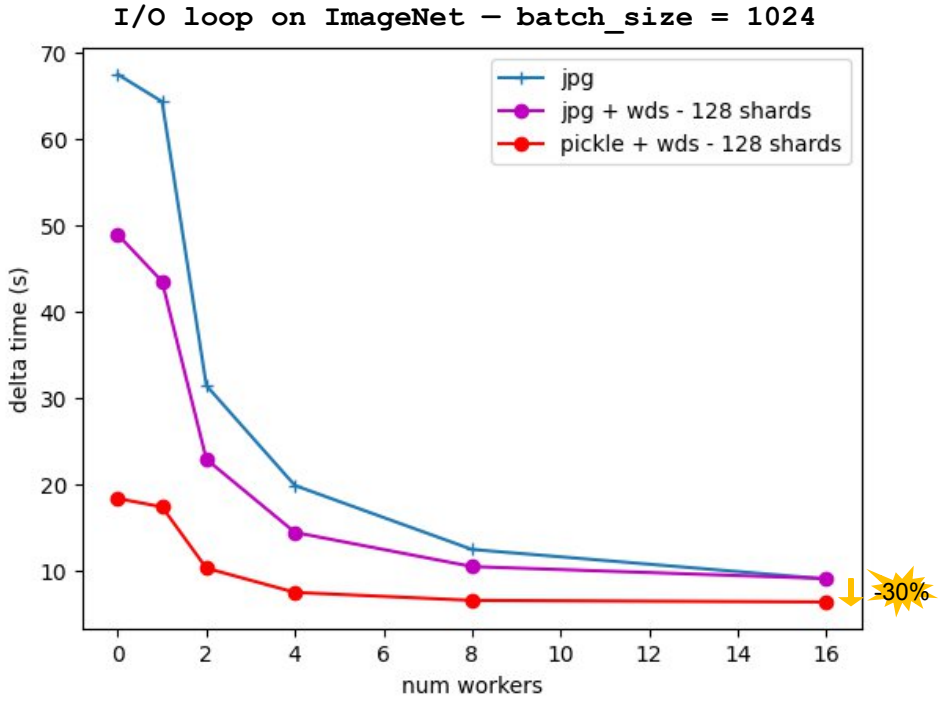
🤔 Does it scale up nicely?

Benchmark 3

Dataset: Same as above

Format: Same as above

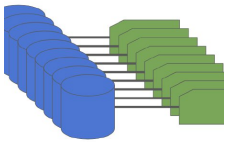
Run: I/O loop while increasing the number of workers



- The WebDataset scales well
- The I/O speedup from sharding diminishes as the number of workers increases

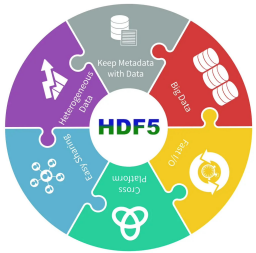
Conclusion on dataset format

- For very large datasets, gathering several samples in shards is **mandatory** to minimize the number of inodes.
- Fewer inodes yield fewer I/Os. This may improve the loading performance but DataLoader standard optimizations (e.g. `num_workers`) are more efficient for this purpose.
- Standard formats already exist like WebDataset, TFRecord, Apache Parquet, HDF5, Zarr,...











General conclusion - Context

jpg

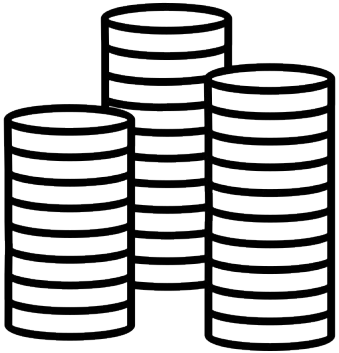


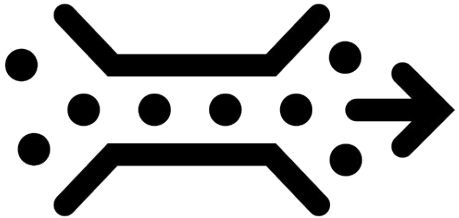
Torch tensor

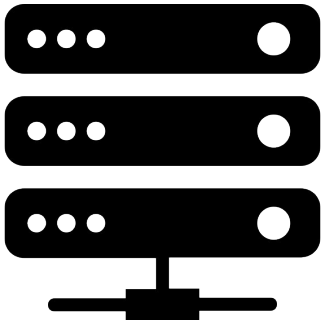
```
tensor([[[144, 145, 145,  ...
         [144, 144, 145,  ...
         [144, 144, 145,  ...
         ...,
         [ 18,  39,  55,  ...
         [  6,  35,  54,  ...
         [  0,  14,  42,  ...
```

CPU
RAM

I/O request + decoding







Storage disks

Interconnection network
OmniPath/InfiniBand

CPU workers



Read/write performance

Shared bandwidth

Decoding performance

General conclusion – Takeaways

Where should I store my dataset?



No automatic deletion

Best I/O performance
Very large quota

Which format for my data?



Fast decoding

Low storage usage

Which format for my dataset?



Map-style dataset
Easy to create and work with

Significantly fewer inodes
Fewer I/Os → potential speedup